PROJECT BRIEF

Wafer Yield Intelligence Programme

Yield forecasting, root-cause analysis and an AI assistant for a semiconductor fab

Client (fictional)	Nexora Semiconductor Manufacturing, Fab Operations & Yield Engineering
Engagement type	Data analytics and AI consulting, 5 one-week Scrum sprints plus Sprint 0
Your role	Lead Data and AI Consultant (you also act as Scrum Master and Developer)
Data provided	semiconductor_yield_forecasting_data.csv: 1,250 wafers, 50 lots, 28 columns, 1 Jan to 19 Feb 2023
Final deliverables	Analysis notebook, yield prediction model, dashboard, AI copilot with prompt library, executive deck

How to use this document. Read it as if it were the statement of work your manager hands you on day one. Sections 1 to 3 explain the client's problem and what they expect. Sections 4 to 6 explain how you will work (Agile sprints). Section 7 is the AI / prompt engineering part. The last section tells you how to turn the project into a portfolio piece.

1. Business context and problem statement

Nexora runs a fab that produces five product families (CPU, GPU, FPGA, ASIC, Memory) across five technology nodes (7nm to 28nm). Every wafer passes through etch, lithography, deposition and implant tools. **Yield** (the share of working dies on a wafer) is the number that decides profit: each percentage point lost is revenue thrown away, and low-yield wafers are only discovered late, after weeks of expensive processing.

The client's yield engineers currently look at spreadsheets and tool dashboards one at a time. They cannot easily say *why* one lot yields 70% and another 31%, and they find out too late to act.

Problem statement

"Nexora's average wafer yield is about 46%, and it swings from 31% to 69% between lots with no clear explanation. Yield engineers need to (1) understand what drives the loss, (2) predict low-yield wafers early enough to intervene, and (3) get answers from the data quickly without waiting for a data scientist."

Key business questions the client wants answered

- Which products, nodes and tools lose the most yield, and is the gap real or just noise?
- Which process and metrology parameters are the strongest yield drivers, and what are their safe operating windows?
- How much of the variation comes from lot-to-lot differences versus wafer-to-wafer differences inside a lot?
- Can we forecast yield before final test, and how early can we raise a warning?
- What is the financial value of closing the gap, and what should we do first?
- Can an AI assistant explain results and answer engineers' questions in plain English?

2. What the client expects from you

In a real engagement the client does not want "a model". They want decisions. Below is what Nexora's Head of Yield Engineering would put in the kickoff email.

#	Expectation	What "done" looks like (acceptance criteria)
E1	**Trusted data.** Confirm the data is fit for analysis	Data dictionary, data-quality log, a clear yield definition, list of open questions for the client
E2	**Explain the yield loss.** Root-cause analysis	Ranked list of drivers with evidence (stats tests, charts); tool matching report; lot vs wafer variance split
E3	**Predict low-yield wafers.** Forecasting model	Model beats a naive baseline on lot-grouped cross-validation; error reported in yield points; feature importance explained
E4	**Actionable recommendations.** Not just charts	Top 3 to 5 actions with expected yield gain and priority; assumptions stated clearly
E5	**Self-serve insight.** Dashboard	Interactive view by product, node, tool and lot; refreshes from a new CSV
E6	**AI assistant.** Natural-language insights	Prompt library, tested on a golden question set, with guardrails against made-up numbers
E7	**Executive story.** Communication	One 8 to 10 slide deck and a 2-page summary a VP can read in 5 minutes

Success metrics

- **Model:** lower mean absolute error (MAE) than a "predict the average" baseline, validated with GroupKFold by lot (so wafers from the same lot never sit in both train and test).
- **Business:** a quantified yield-improvement opportunity, for example "bringing the worst tool group to the median adds X yield points, worth Y".
- **Adoption:** a yield engineer can answer a new question in under 2 minutes using the dashboard or the AI assistant.

3. What is in your data (from the actual file)

Group	Columns	What they tell you
Identifiers and context	lot_id, wafer_id, product_type, technology_node, process_date	Which wafer, what product, which node, when. 50 lots x 25 wafers
Tools	etch_tool (5), litho_tool (4), deposition_tool (3), implant_tool (3)	Equipment used; basis for tool-matching analysis
Process settings	etch_rate, pressure, temperature, exposure_time, focus_offset, dose, deposition_rate, implant_energy, tilt_angle	Recipe and tool conditions during processing
Inline metrology	thickness_uniformity, critical_dimension, oxide_thickness, resistivity	Physical measurements taken on the wafer
Defects	defect_count, defect_density	Particle and pattern defects found by inspection
Electrical test	vth, leakage_current, resistance	Device-level performance
Target	yield (0.15 to 0.98)	The outcome to explain and forecast

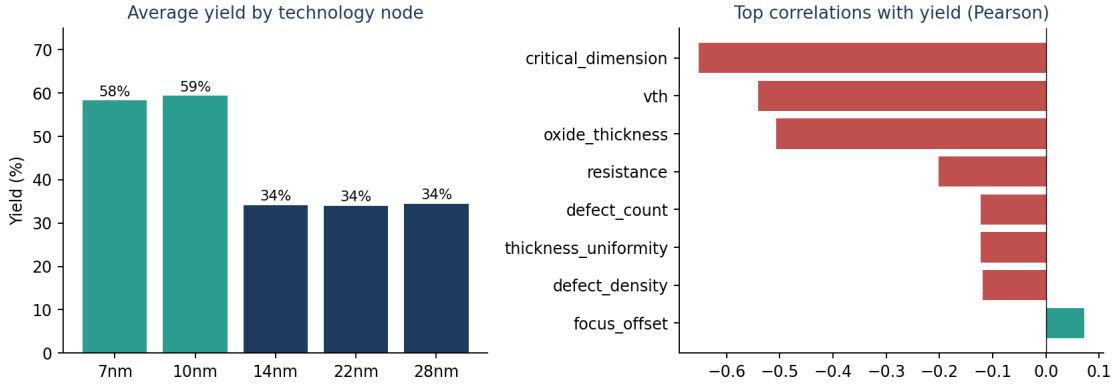


Figure 1. Left: 7nm and 10nm wafers show much higher yield than 14, 22 and 28nm in this file. Right: critical dimension, threshold voltage (vth) and oxide thickness have the strongest linear link to yield.

First observations to bring to the client

- **Critical dimension (r = -0.65), vth (-0.54) and oxide thickness (-0.51)** are the strongest signals. Most raw process settings (pressure, temperature, etch rate, exposure time) show almost no linear relationship with yield on their own, so interactions or tool and node effects may matter more.
- **Variation inside lots is large:** roughly 47% of total yield variance is wafer-to-wafer within the same lot, so lot-level averages alone will not explain everything.
- **Tool differences exist:** LITHO_01 averages about 52.5% against 40.6% for LITHO_02; IMP_03 about 52.3% against 42.0% for IMP_02. These need statistical testing because tools are also mixed with different products and nodes (confounding).
- **Surprising pattern:** the more advanced 7nm and 10nm nodes yield about 58 to 59% while 14 to 28nm sit near 34%. In real fabs mature nodes usually yield higher. A good consultant raises this with the client instead of assuming it is correct (see risks).
- **Small time window:** only about 7 weeks of data, so long-term drift or seasonality cannot be studied. Say this clearly in your findings.

Important modelling point for the client. Critical dimension, vth, oxide thickness and defect data are measured *after* processing. They are great for explaining why yield dropped, but they are not available for an early forecast. You will therefore build **two models**: an **Early-Warning model** (tool, product, node and process settings only) and a **Diagnostic model** (adds metrology and electrical tests). Mixing them would give a model that looks excellent but is useless in practice (data leakage).

4. Scope and analytical approach

In scope	Out of scope (say this to the client upfront)
Cleaning and profiling the supplied CSV Root-cause analysis and tool matching Two yield-prediction models Dashboard and executive deck AI assistant prototype with prompt library	Changing live fab recipes or tool settings Die-level or wafer-map image analysis Real-time integration with MES or SPC systems Production deployment and monitoring

Analysis workstreams

Workstream	Techniques to practise	Output
A. Data quality	Pandas profiling, outlier checks (IQR, z-score), range sanity checks, duplicate wafer keys, units	Data-quality log, data dictionary
B. Descriptive and SPC	Yield by product, node, tool; control charts per lot; Pareto of loss; yield distribution and low-yield threshold (for example below 30%)	Baseline report
C. Root cause	ANOVA or Kruskal-Wallis by tool, Tukey post-hoc, partial correlation, variance components (lot vs wafer), tool x node cross-tabs to control confounding	Ranked drivers, tool matching report
D. Prediction	Linear baseline, Random Forest, Gradient Boosting (LightGBM or XGBoost), GroupKFold by lot, MAE and R-squared, SHAP for explanation	Early-Warning and Diagnostic models
E. Recommendation	What-if analysis, safe operating windows, opportunity sizing (yield points x wafer starts x revenue per wafer, assumptions stated)	Action plan with value
F. AI assistant	Prompt engineering, structured outputs, evaluation set, guardrails	Prompt library and demo

Tech stack suggestion (all free)

Python (pandas, scipy, statsmodels, scikit-learn, lightgbm, shap), Jupyter, Streamlit or Power BI for the dashboard, GitHub for version control, an LLM (Claude) for the AI assistant, and a simple Kanban board (Trello, Jira or GitHub Projects) for the backlog.

5. Working the Agile (Scrum) way

Scrum is a way of delivering in short cycles (sprints), showing something usable at the end of each one, and adapting based on feedback. This project uses **one-week sprints** so you see the full cycle five times. If you work part-time, stretch each sprint to two weeks.

Roles (you play several, which is fine for learning)

Scrum role	In this project	Tip
Product Owner	The client (Head of Yield Engineering). Owns priorities and says what "good" is	Ask Claude to role-play the client in your Sprint Review and give feedback
Scrum Master	You. Keeps the process, removes blockers	Keep a short impediments list
Developers	You. Analysis, modelling, dashboard, prompts	Limit work in progress to 2 items at a time

Ceremonies and artefacts (per sprint)

Event or artefact	When and how long	Purpose
Sprint Planning	Day 1, 45 min	Pick sprint goal and backlog items you can finish; split into tasks
Daily Stand-up	Every day, 5 min (write 3 lines)	What I did, what I will do, what blocks me
Sprint Review	Last day, 30 min	Demo the increment to the "client" and collect feedback
Retrospective	Last day, 15 min	What went well, what to improve, one action for next sprint
Product Backlog	Living list	All user stories, ordered by value
Definition of Done	Fixed checklist	Code in GitHub, notebook runs top to bottom, findings written in plain English, peer or self review done

Write requirements as user stories: *"As a [role], I want [need], so that [benefit]"*, each with acceptance criteria. Estimate with story points (1, 2, 3, 5, 8) and track a burndown chart.

6. Sprint plan

Sprint	Goal	Key user stories	Increment shown at review
Sprint 0 2 to 3 days	Set up and align	Kickoff notes, problem framing, backlog, repo and board, Definition of Done, risks list	Agreed problem statement and backlog
Sprint 1 Data understanding	Know and trust the data	**US1** As a yield engineer I want a data dictionary so I know what each column means. **US2** As a client I want data issues listed so I know what to trust. **US3** As an analyst I want yield defined (per wafer, low-yield threshold) so we speak one language.	Profiling notebook, data-quality log, 1-page data summary, questions for client
Sprint 2 Root cause	Explain where yield is lost	**US4** I want yield by product, node and tool with significance tests. **US5** I want the lot vs wafer variance split. **US6** I want control charts to spot out-of-control lots. **US7** I want driver ranking for metrology and process settings.	Root-cause report: top drivers, tool matching table, Pareto and SPC charts
Sprint 3 Prediction	Forecast low-yield wafers	**US8** I want a baseline model to know the bar to beat. **US9** I want an Early-Warning model using pre-test data only. **US10** I want a Diagnostic model with metrology, and SHAP explanations. **US11** I want honest validation (GroupKFold by lot).	Two models with metrics, feature importance, limitations page
Sprint 4 AI assistant	Make insights conversational	**US12** I want to ask questions in plain English and get answers grounded in the data. **US13** I want a short narrative for each lot or wafer. **US14** I want guardrails so the AI never invents numbers.	Prompt library v1 to v3, evaluation sheet, demo (Section 7)
Sprint 5 Deliver	Turn insight into decisions	**US15** I want a dashboard to explore yield. **US16** I want a ranked action plan with value estimate. **US17** I want an executive deck and 2-page summary.	Dashboard, final deck, handover pack, final retrospective

Example of a full user story (copy this format)

US9, Early-Warning model. As a yield engineer, I want a model that predicts wafer yield using only information available before electrical test, so that I can hold or re-route risky lots.
Acceptance criteria: (1) uses only tool, product, node and process-setting columns; (2) validated with GroupKFold by lot; (3) MAE is lower than the "predict the mean" baseline; (4) top 5 features explained in plain English; (5) limitations documented.

Suggested daily rhythm for a one-week sprint

- Day 1: planning, set up tasks. Days 2 to 4: build, with a 5-minute stand-up note each morning.
- Day 5: Sprint Review (demo to yourself or to Claude playing the client), retrospective, update backlog.

7. AI and prompt engineering: the Yield Copilot (Sprint 4)

Core idea: the machine-learning model does the prediction; the language model (LLM) does the explaining. Never let the LLM "guess" yield. Give it verified numbers from your analysis and let it turn them into clear language, answers and next steps. This division of labour is how real enterprise AI assistants are built.

Use case	Input to the LLM	Output
Insight narrator	JSON with computed stats: yield by tool, top drivers, model metrics	Executive summary in 5 bullet points
Wafer triage assistant	One wafer's parameters, lot and tool averages, model prediction and SHAP drivers	Likely causes, confidence, 3 checks the engineer should run
Data Q&A	User question plus data dictionary and a few sample rows	Pandas code (run by you), then a plain-English answer from the result
Stakeholder rewriter	Same finding, different audience tag	Version for engineer, manager or VP

Prompt anatomy to practise (R-C-D-T-C-F)

Part	Meaning	Example
Role	Who the model is	Senior semiconductor yield engineer
Context	Business and data background	Fab, 5 nodes, yield is share of good dies
Data	Verified facts only	JSON block of results
Task	Exactly what to do	Summarise top 3 yield drivers
Constraints	Rules and guardrails	Use only supplied numbers; say "not in data" otherwise
Format	Structure of the answer	JSON or 5 bullets, max 120 words

Starter system prompt (improve it across v1, v2, v3)

```
ROLE: You are a senior semiconductor yield engineer advising a fab's operations team.
CONTEXT: Yield = fraction of good dies per wafer (0 to 1). Data covers 50 lots,
5 product types, 5 technology nodes, Jan-Feb 2023.
DATA: <<verified_results_json>>
TASK: Explain the main causes of low yield and recommend next actions.
CONSTRAINTS:
- Use ONLY numbers in DATA. If something is not in DATA, reply "Not in the data".
- Separate FACT (from data) from HYPOTHESIS (your engineering judgement).
- Mention sample size when comparing tools or nodes.
- Do not claim causation from correlation; suggest a test instead.
FORMAT: JSON {"summary": str, "facts": [str], "hypotheses": [str],
"next_checks": [str], "confidence": "low|medium|high"}
```

How to evaluate your prompts (this is what separates engineering from guessing)

- Build a **golden set of 20 questions** with correct answers you computed yourself in pandas (for example "Which litho tool has the lowest average yield?").
- Score every prompt version on **faithfulness** (numbers match), **completeness**, **actionability** and **format compliance** (1 to 5 each). Keep a version log: v1 baseline, v2 adds constraints, v3 adds few-shot examples.
- Test **failure cases on purpose**: ask about a column that does not exist, ask for a prediction for 3nm, push the model to claim causation. A good prompt refuses or flags uncertainty.
- Techniques to try: few-shot examples, chain-of-thought-style step planning (ask for reasoning in a hidden field), structured JSON output, and self-check ("verify each number against DATA before answering").

8. Risks, assumptions and open questions for the client

Risk or question	Why it matters	Mitigation
7nm and 10nm yield higher than 14 to 28nm	Opposite of normal industry pattern; may reflect product mix, yield definition or synthetic data	Ask client to confirm; analyse within product type; state it as an assumption
Confounding (tool, node and product are mixed)	A tool can look bad only because it runs hard products	Compare tools within the same node and product; use regression with controls
Data leakage	Post-process metrology cannot predict early	Two-model design; document available-at-prediction-time for each column
Short time window (7 weeks)	Cannot test drift or seasonality; time-based validation is limited	Use GroupKFold by lot; flag as limitation; recommend collecting more data
Outliers and sensor errors	Could distort models	Review with client before removing; never delete silently
LLM hallucination	Wrong numbers damage trust	Ground in verified JSON, golden-set testing, refusal rules

9. Turning this into a portfolio project

- **GitHub repo** with folders: data, notebooks, src, dashboard, prompts, docs. Include a README that starts with the business problem, not the code.
- **Sprint evidence:** backlog screenshots, sprint goals, review notes and retrospectives. Interviewers love seeing that you actually practised Agile.
- **One-page case study** in the format: Problem, Approach, Findings, Impact, What I would do next.
- **Talk track** for interviews: "I worked as a consultant for a fictional fab. The key finding was X, the model reduced error by Y versus baseline, and I added an LLM copilot with a tested prompt library."
- **Honesty note:** label the client and data as fictional or synthetic. Credibility comes from method and clear thinking, not from pretending it was real.

Suggested next step: start Sprint 0 today. Create the repo and board, copy the backlog from Section 6, and write your own one-paragraph version of the problem statement. When you finish Sprint 1, come back and I can review your data-quality log, help with the models, or play the client in your Sprint Review.